

Agenda: Day 2

- Recap and Solution for Day 1
- What is Static Analysis?
- Code Patching: A Magic Trick
- Intro to Ghidra
- Intro to IDA
- Discussion and Resources

Recap and Solution for Day 1

- We discussed memory hierarchy and the stack
- We talked about x86-64 assembly and looked at some basic instructions
- Solution for Day 1 code snippet

What is Static Analysis?

Static Analysis involves trying to understand the program without executing it. This involves disassembling and decompiling the binary and reading the assembly instructions and pseudocode.

The goal is to get a sense of what the program is doing before jumping into dynamic analysis. For simple programs, it might be possible to completely reverse engineer the binary with static analysis alone.

Code Patching: A Magic Trick

What if you could modify a compiled binary and convince it to do something it wasn't designed to do? Turns out, you can!

Code patching involves carefully modifying certain bytes in the binary to control the program flow or trigger a certain condition.

Intro to Ghidra

<https://github.com/NationalSecurityAgency/ghidra>

Open the binary inside Ghidra

Program Trees

ghidra_example

.bss

.data

.got.plt

.got

dynamic

Program Tree

Symbol Tree

Imports

Exports

Functions

Labels

Classes

Namespaces

Filter:

Data Type Manager

Data Types

BuiltInTypes

ghidra_example

generic_clib_64

Filter:

Listing: ghidra_example_1

* FUNCTION

undefined main()
 <UNASSIGNED> <RETURN>
 Stack[-0xc]:4 local_c
main

XREF

00101189 55 PUSH RBP
0010118a 48 89 e5 MOV RBP,RSP
0010118d 48 83 ec 10 SUB RSP,0x10
00101191 c7 45 fc MOV dword ptr [RBP + local_c],0x4
 04 00 00 00
00101198 83 7d fc 02 CMP dword ptr [RBP + -0x4],0x2
0010119c 75 07 JNZ LAB_001011a5
0010119e e8 18 00 CALL flag
 00 00
001011a3 eb 0f JMP LAB_001011b4

LAB_001011a5
001011a5 48 8d 05 LEA RAX,[s_no_flag_:(_00102004)]
 50 00 00 00

XREF

Decompil...

1

2 /* WARNING: Removing unreachable block (ra

3

4 undefined8 main(void)

5

6 {

7 puts("no flag :(");

8 return 0;

9 }

10

Console - Scripting

00101189

main

PUSH RBP

Patch the specific instruction by Right Click > Patch Instruction

Program Trees

ghidra_example

.bss

.data

.got.plt

.got

dynamic

Program Tree

Symbol Tree

Imports

Exports

Functions

Labels

Classes

Namespaces

Filter:

Data Type Manager

Data Types

BuiltInTypes

ghidra_example

generic_clib_64

Filter:

Listing: ghidra_example_1

* FUNCTION

undefined main()
<UNASSIGNED> <RETURN>
Stack[-0xc]:4 local_c
main

XREF

00101189	55	PUSH	RBP
0010118a	48 89 e5	MOV	RBP,RSP
0010118d	48 83 ec 10	SUB	RSP,0x10
00101191	c7 45 fc	MOV	dword ptr [RBP + local_c],0x4
	04 00 00 00		
00101198	83 7d fc 02	CMP	dword ptr [RBP + -0x4],0x2
0010119c	75 07	JNZ	LAB_001011a5
0010119e	e8 18 00	CALL	flag
	00 00		
001011a3	eb 0f	JMP	LAB_001011b4
		LAB_001011a5	XREF
001011a5	48 8d 05	LEA	RAX,[s_no_flag_:(_00102004)]
	50 00 00 00		

Console - Scripting

Decompil...

1

2 /* WARNING: Removing unreachable block (ra

3

4 undefined8 main(void)

5

6 {

7 puts("no flag :(");

8 return 0;

9 }

10

00101198

main

CMP dword ptr [RBP + -0x4]...

Modify the bytes in the instruction

CodeBrowser: re-bootcamp:/ghidra_example_1

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help

Program Trees

- ghidra_example
 - .bss
 - .data
 - .got.plt
 - .got
 - dynamic

Program Tree x

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces

Filter:

Data Type Manager

- Data Types
 - BuiltInTypes
 - ghidra_example
 - generic_clib_64

Filter:

Listing: ghidra_example_1

```
*****
*                               FUNCTION
*****

undefined main()
<UNASSIGNED> <RETURN>
Stack[-0xc]:4 local_c
main

00101189 55          PUSH      RBP
0010118a 48 89 e5     MOV       RBP,RSP
0010118d 48 83 ec 10  SUB      RSP,0x10
00101191 c7 45 fc     MOV      dword ptr [RBP + local_c],0x4
00101198 83 7d fc 02  CMP      dword ptr [RBP + -0x4],0x4
0010119c 75 07
0010119e e8 18 00
001011a3 eb 0f
001011a5 48 8d 05     LAB_001011a5: MOV      RAX,0x5
001011a8 50 00 00 00
```

XREF

Decompil...

```
1
2 /* WARNING: Removing unreachable block (ra
3
4 undefined8 main(void)
5
6 {
7     puts("no flag :(");
8     return 0;
9 }
10
```

Console - Scripting

CMP dword ptr [RBP + -0x4]...

Export the program as a binary file

The screenshot shows the Ghidra IDE interface with the 'Export ghidra_example_1' dialog box open. The dialog box has the following fields and options:

- Format: Original File
- Output File: /home/shru/re-bootcamp-ulgc/ghidra_example_1
- Selection Only: ☐
- Buttons: OK, Cancel, Options...

The background shows the Ghidra IDE with the following components:

- Program Trees:** ghidra_example, .bss, .data, .got.plt, .got, dynamic.
- Symbol Tree:** Imports, Exports, Functions, Labels, Classes, Namespaces.
- Data Type Manager:** Data Types, BuiltInTypes, ghidra_example, generic_clib_64.
- Listing: ghidra_example_1:** Shows assembly code for the 'main' function. The code includes a warning about an unreachable block, a call to 'main(void)', and a return statement. The assembly code is as follows:

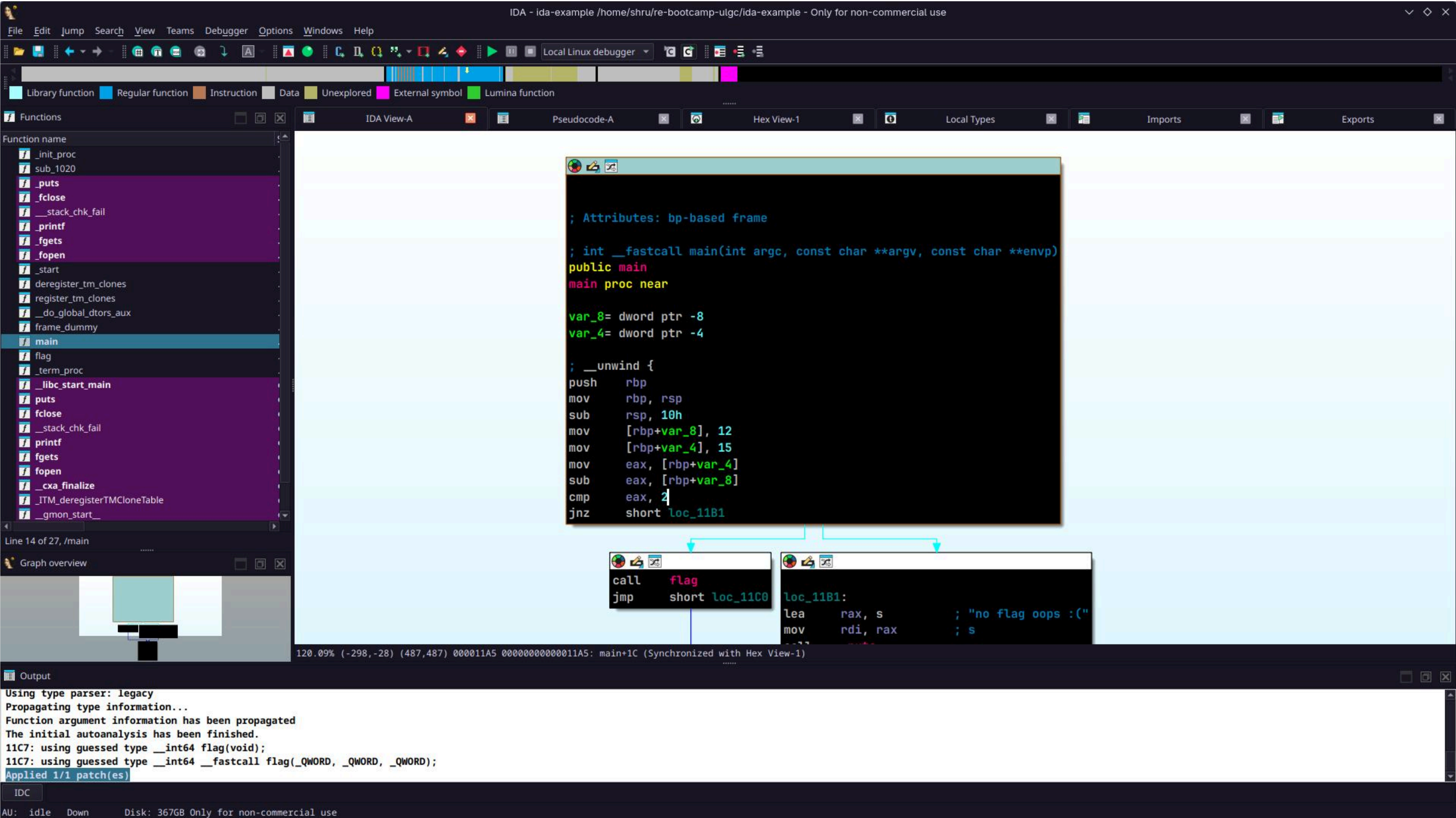
```
*****  
* FUNCTION  
*****  
undefined main()  
undefined4 Stack[-0xc]:4 local_c  
main  
001011a3 eb 0f JMP LAB_001011b4  
LAB_001011a5  
001011a5 48 8d 05 LEA RAX, [s_no_flag_: (_00102004)]  
50 00 00 00
```
- Decompil...:** Shows the decompiled C code for the 'main' function:

```
1  
2 /* WARNING: Removing unreachable block (ra  
3  
4 undefined8 main(void)  
5  
6 {  
7     flag();  
8     return 0;  
9 }  
10
```
- Console - Scripting:** Empty console area.
- Status Bar:** Shows the current instruction address (00101198), function name (main), and instruction (CMP dword ptr [RBP + -0x4]...).

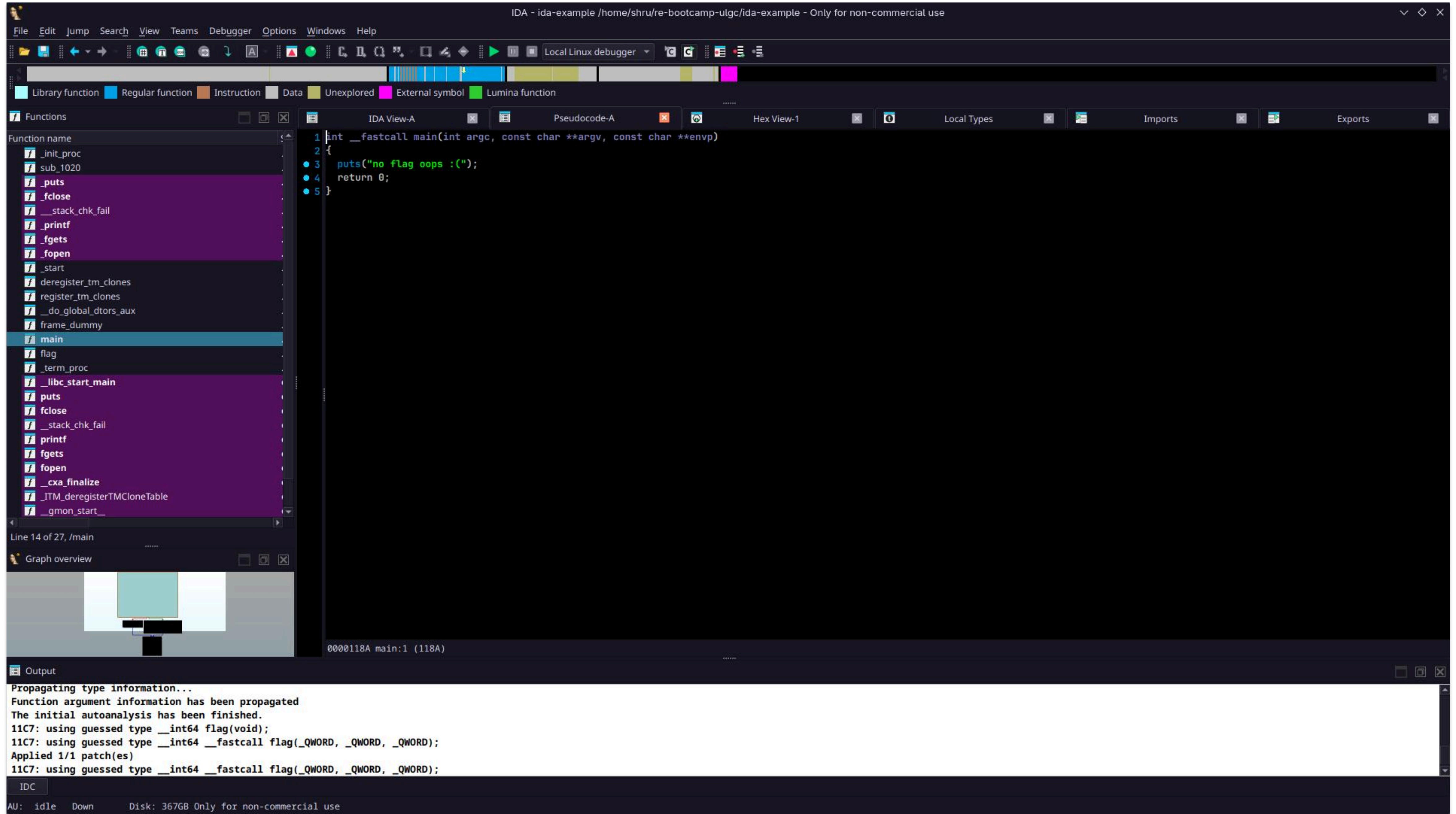
Intro to IDA

<https://hex-rays.com/ida-free>

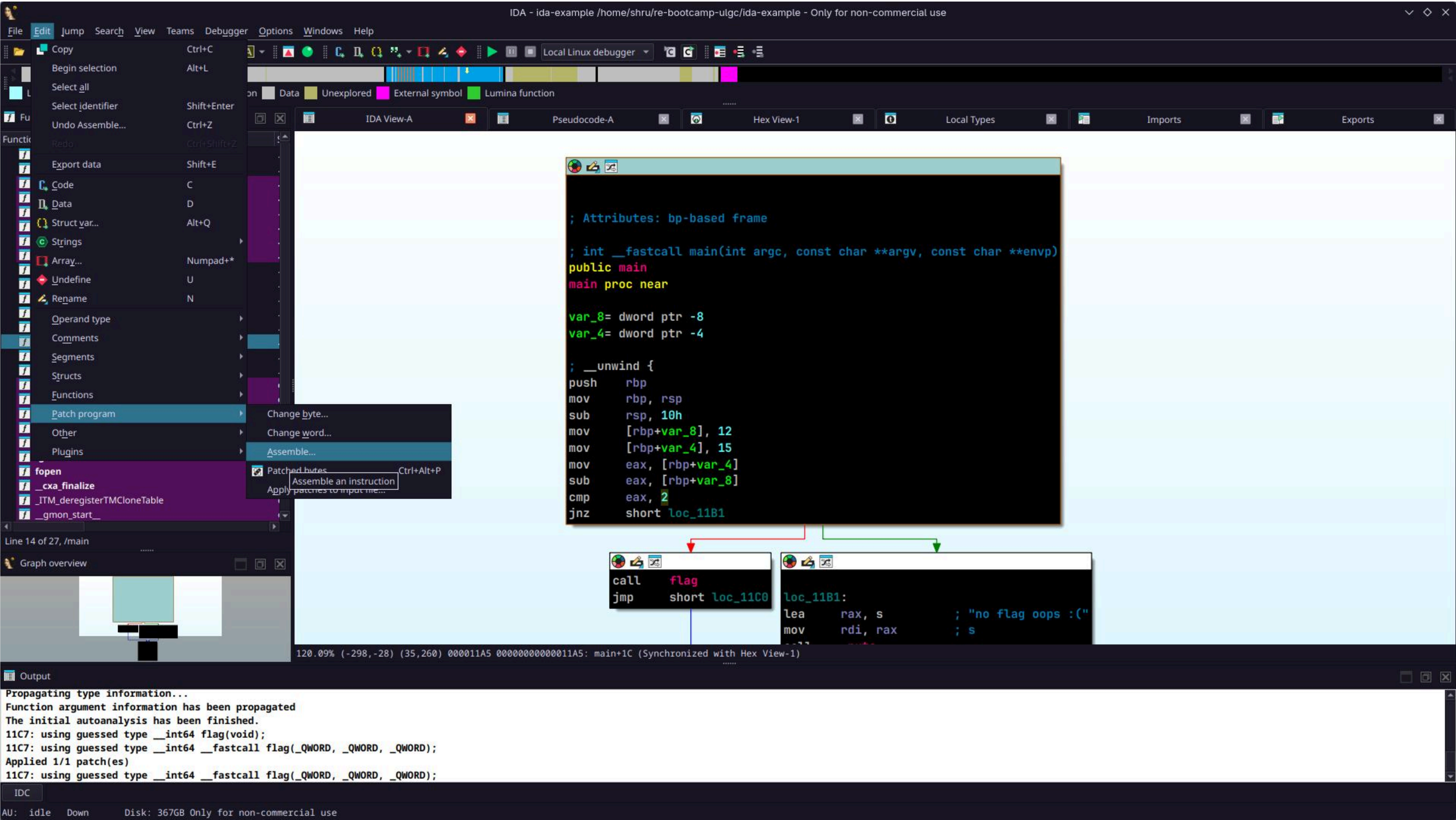
Open the binary inside IDA Free



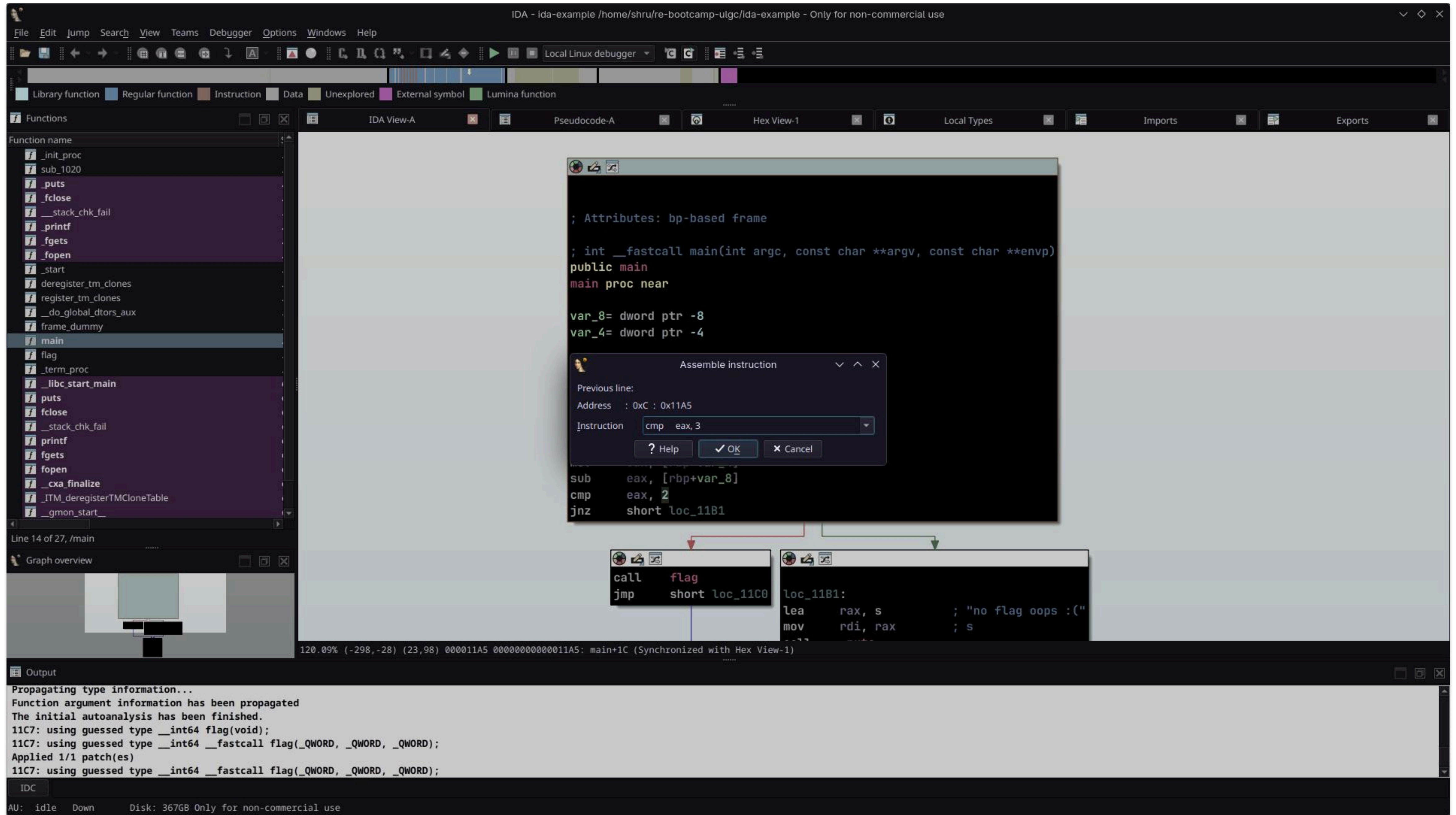
Check the pseudocode with Views > Open Subviews > Generate Pseudocode



Place the cursor on the instruction and open Edit > Patch Program > Assemble



Patch the instruction by modifying specific bytes



Export the program with Edit > Patch program > Apply patches to input file

The screenshot shows the IDA Pro interface with the 'Edit' menu open, highlighting the 'Patch program' option. A submenu is visible with the following options: 'Change byte...', 'Change word...', 'Assemble...', 'Patched bytes' (with a keyboard shortcut 'Ctrl+Alt+P'), and 'Apply patches to input file...'. The 'Apply patches to input file...' option is highlighted. Below the menu, a dialog box titled 'Apply patches to input file' is shown, containing the following assembly code:

```
; Attributes: bp-based frame

; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

var_8= dword ptr -8
var_4= dword ptr -4

; __unwind {
push    rbp
mov     rbp, rsp
sub     rsp, 10h
mov     [rbp+var_8], 12
mov     [rbp+var_4], 15
mov     eax, [rbp+var_4]
sub     eax, [rbp+var_8]
cmp     eax, 3
jnz     short loc_11B1

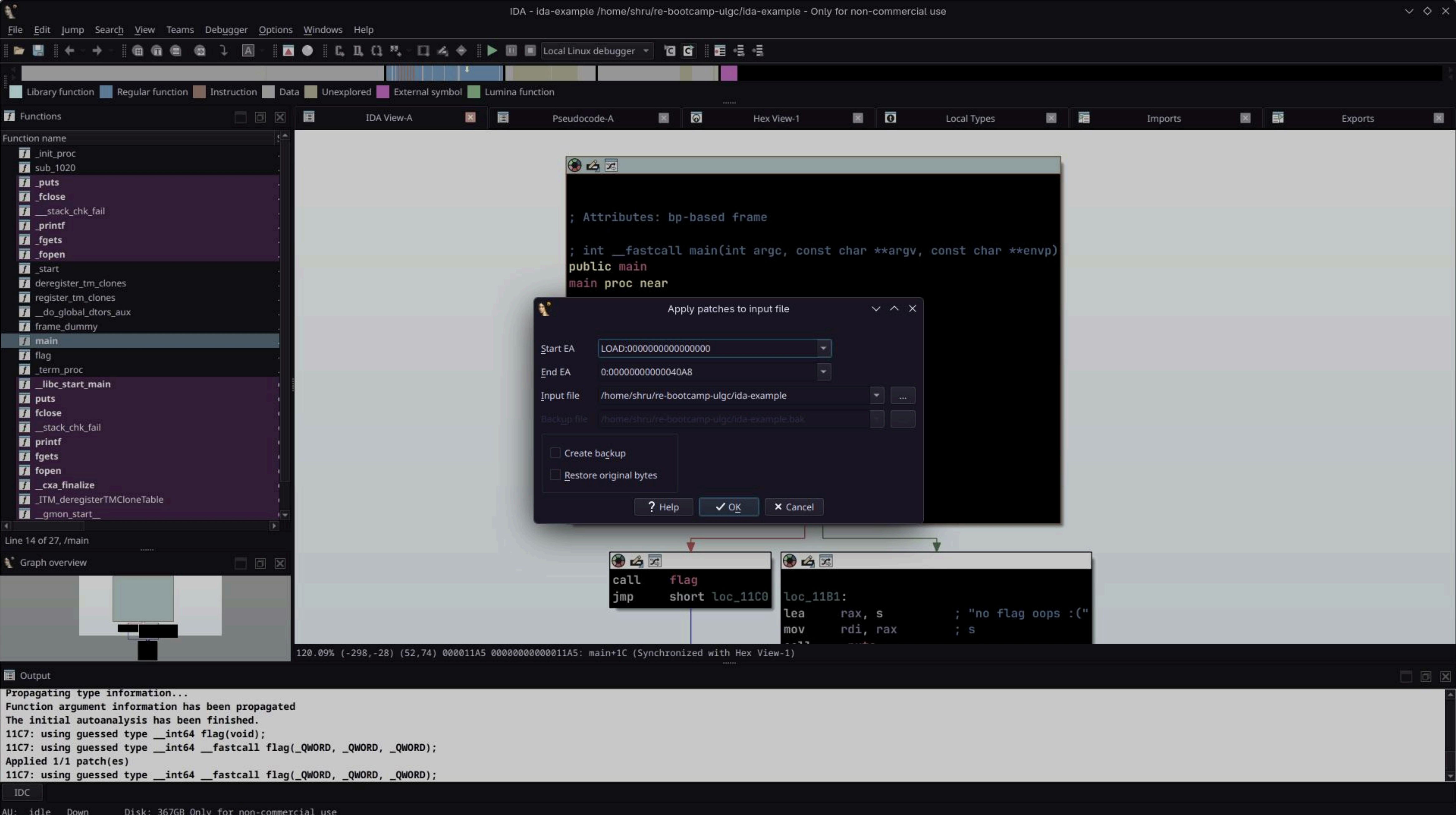
call    flag
jmp     short loc_11C0

loc_11B1:
lea     rax, s           ; "no flag oops :("
mov     rdi, rax          ; s
```

The dialog box also shows a 'call flag' instruction and a 'jmp short loc_11C0' instruction. The 'Apply patches to input file' button is highlighted. The background shows the IDA Pro interface with the 'Edit' menu open, the 'Patch program' option selected, and the 'Apply patches to input file' dialog box. The output window at the bottom shows the following text:

```
Propagating type information...
Function argument information has been propagated
The initial autoanalysis has been finished.
11C7: using guessed type __int64 flag(void);
11C7: using guessed type __int64 __fastcall flag(_QWORD, _QWORD, _QWORD);
Applied 1/1 patch(es)
11C7: using guessed type __int64 __fastcall flag(_QWORD, _QWORD, _QWORD);
```

Save the program



Discussion and Resources

<https://vickieli.dev/binary%20exploitation/intro-to-reverse-engineering/>

<https://vickieli.dev/binary%20exploitation/intro-to-binary-patching/>

<https://app.letsdefend.io/training/lessons/introduction-to-ida>